

IssueFlow Project Explanation Guide

A step-by-step study guide for understanding and explaining the Java Spring Boot homework project.

1. Big Picture

IssueFlow is a backend REST API. It does not render a visual website. Instead, it waits for HTTP requests from a client such as Thunder Client, Postman, curl, or another frontend application.

The backend receives requests, checks security, validates input, applies business rules, reads or writes database records, and returns JSON responses.

```
Client -> Controller -> Service -> Repository -> Database
```

Example flow: Thunder Client sends POST /tickets. TicketController receives the request. TicketService applies ticket rules. TicketRepository saves the ticket. PostgreSQL stores it. TicketResponse returns JSON to the client.

2. Application Startup

The main class is IssueFlowApplication.java. When you run mvnw.cmd spring-boot:run, Spring Boot starts the application.

- Tomcat starts on port 8080.
- Spring connects to PostgreSQL.
- Spring scans controllers, services, repositories, and configuration classes.
- JPA/Hibernate creates or updates database tables.
- The overdue-ticket scheduler starts running in the background.

```
Tomcat started on port 8080<br/>Started IssueFlowApplication
```

Those lines mean the backend is running successfully at http://localhost:8080.

3. Security And Authentication

Most endpoints are protected. The public endpoints are POST /users and POST /auth/login. Everything else requires a JWT token.

Authentication flow:

- Create a user with POST /users.
- Login with username and password using POST /auth/login.
- The backend validates the password and returns a signed JWT token.
- For protected requests, send Authorization: Bearer <token>.
- JwtAuthenticationFilter validates the token and sets the authenticated user in Spring Security.

```
Authorization: Bearer eyJhbGciOiJIUzI1NiJ9...
```

Important files: SecurityConfig.java, AuthController.java, AuthService.java, JwtService.java, JwtAuthenticationFilter.java.

4. User Management

Users represent people in the system. Each user has a username, email, password, full name, and role. Valid roles are ADMIN and DEVELOPER.

Passwords are stored as BCrypt hashes, not plain text.

Create-user flow:

- UserController receives POST /users.
- UserService checks that username and email are unique.
- UserService hashes the password.
- UserRepository saves the user.
- AuditLogService records CREATE USER.
- UserResponse returns user data without the password.

Important files: User.java, UserController.java, UserService.java, UserRepository.java, CreateUserRequest.java, UserResponse.java.

5. Project Management

A project groups tickets. Each project has a name, description, owner, and deleted flag.

```
{<br/> "name": "IssueFlow Demo",<br/> "description": "Testing homework requirements",<br/> "ownerId": 16<br/>}
```

Create-project flow:

- ProjectController receives POST /projects.
- ProjectService finds the owner user by ownerId.
- ProjectService creates and saves the project.
- AuditLogService records CREATE PROJECT.
- ProjectResponse returns id, name, description, ownerId, and deleted.

Projects use soft delete. Deleting a project sets deleted = true instead of removing the row. Normal project responses hide deleted projects. Admin-only endpoints can list and restore them.

6. Ticket Management

Tickets are the core work items. A ticket belongs to exactly one project and can optionally have an assignee.

- Valid statuses: TODO, IN_PROGRESS, IN_REVIEW, DONE.
- Valid priorities: LOW, MEDIUM, HIGH, CRITICAL.
- Valid types: BUG, FEATURE, TECHNICAL.

Create-ticket flow:

- TicketController receives POST /tickets.
- TicketService finds the project.
- TicketService resolves the assignee. If no assigneeId is sent, auto-assignment runs.
- TicketRepository saves the ticket.
- AuditLogService records CREATE TICKET.

- If auto-assignment happened, AuditLogService records AUTO_ASSIGN.
- TicketResponse returns the created ticket.

7. Auto-Assignment

When a ticket is created without assigneeId, the system automatically chooses a developer.

- It checks all users with role DEVELOPER.
- It counts each developer's non-DONE tickets in the same project.
- It picks the developer with the lowest open-ticket count.
- If there is a tie, the oldest registered developer wins.
- The assignment is recorded in the audit log with action AUTO_ASSIGN and actor SYSTEM.

The workload endpoint is GET /projects/{projectId}/workload. It returns developers sorted by openTicketCount ascending. For tied workloads, older users remain first.

8. Ticket Update Rules

Ticket lifecycle rules protect the workflow.

- Status can only move forward: TODO -> IN_PROGRESS -> IN_REVIEW -> DONE.
- Backward transitions are rejected.
- A DONE ticket cannot be updated.
- A ticket cannot move to DONE if it has unresolved blockers.

Tickets use @Version optimistic locking, so JPA can detect conflicting simultaneous updates.

9. Ticket Dependencies

A ticket can be blocked by another ticket. Example: ticket 10 is blocked by ticket 9.

```
POST /tickets/10/dependencies<br/>{ "blockedBy": 9 }
```

Dependency rules: both tickets must exist, both must belong to the same project, and a ticket cannot move to DONE while any blocker is not DONE.

Important files: TicketDependency.java, TicketDependencyController.java, TicketDependencyService.java, TicketDependencyRepository.java.

10. Comments

Users can add comments to tickets. Comments include content, author, ticket, timestamps, and version.

```
POST /tickets/{ticketId}/comments<br/>{<br/> "authorId": 16,<br/> "content": "Please check this @dev1"<br/>}
```

Comment flow:

- CommentController receives the request.
- CommentService finds the ticket and author.
- CommentRepository saves the comment.

- MentionService refreshes mentions from the comment content.
- AuditLogService records CREATE COMMENT.
- CommentResponse returns the comment and mentionedUsers.

Comments also use @Version for simultaneous edit protection.

11. Mentions

Mentions are created when comment content contains @username. Matching is case-insensitive.

- MentionService parses @username from the comment text.
- It finds the matching user case-insensitively.
- It stores a Mention record.
- Comment responses include mentionedUsers.
- On comment update, mentions are recalculated: removed mentions are deleted and new mentions are created.

```
GET /users/{userId}/mentions?page=1&pageSize=20
```

Important files: Mention.java, MentionService.java, MentionRepository.java, MentionPageResponse.java, MentionedUserResponse.java.

12. Attachments

Users can upload files to tickets. The service validates file size and content type before saving.

- Maximum file size: 10 MB.
- Allowed content types: image/png, image/jpeg, application/pdf, text/plain.
- Invalid file types or oversized files are rejected.
- Attachment metadata and bytes are persisted.
- Upload and delete actions are audited.

Important files: Attachment.java, AttachmentController.java, AttachmentService.java, AttachmentRepository.java, AttachmentResponse.java.

13. CSV Import And Export

Tickets can be exported and imported using CSV.

- GET /tickets/export?projectId={id} returns CSV data.
- POST /tickets/import?projectId={id} accepts a CSV file and creates tickets.
- The service returns a summary with created count, failed count, and row errors.
- Apache Commons CSV is used so commas and quotes inside values are handled correctly.

Important file: TicketCsvService.java.

14. Soft Delete

Projects and tickets are soft-deleted. That means deleted = true instead of physically removing database rows.

- Normal GET endpoints hide deleted projects and tickets.

- Admin-only endpoints list deleted records.
- Admin-only restore endpoints set deleted back to false.
- Permanent delete is not exposed through the API.

```
GET /tickets/deleted?projectId=10<br/>POST /tickets/{id}/restore<br/>GET /projects/deleted<br/>POST /projects/{id}/restore
```

15. Audit Logs

Every state-changing action records an audit log. This creates a history of what happened in the system.

- CREATE USER, CREATE PROJECT, CREATE TICKET.
- UPDATE and DELETE actions.
- AUTO_ASSIGN for automatic ticket assignment.
- AUTO_ESCALATE for scheduled overdue-ticket escalation.

Audit logs store entityType, entityId, action, actor, details, and timestamp. The endpoint GET /audit-logs supports filters such as entityType, entityId, action, and actor.

16. Auto-Escalation

A background scheduler checks overdue tickets. This is why Hibernate select statements repeat in the running terminal.

- It checks tickets where status is not DONE, deleted is false, and dueDate is before now.
- LOW becomes MEDIUM.
- MEDIUM becomes HIGH.
- HIGH becomes CRITICAL.
- If already CRITICAL and still overdue, overdue is set to true.
- The scheduler does not change ticket status.

Important file: TicketEscalationService.java.

17. Error Handling

The project has centralized API error handling. Instead of exposing raw Java stack traces to clients, it returns useful HTTP errors.

- BadRequestException for invalid user input or business-rule violations.
- ResourceNotFoundException when an entity does not exist.
- ApiExceptionHandler maps exceptions to HTTP responses.
- ApiError defines the error response shape.

18. Testing

Tests live in IssueFlowApplicationTests.java. They use an H2 in-memory database, not the real PostgreSQL database. This makes tests safe and repeatable.

- Spring context loads.

- Project stores required owner.
- Mentions are case-insensitive.
- Mentions refresh after comment update.
- A comment must belong to the ticket path used in the request.
- Ticket status cannot move backward.
- Blocked tickets cannot move to DONE.
- DONE tickets cannot be updated.

19. How To Explain The Architecture

"I used a layered Spring Boot architecture. Controllers expose HTTP endpoints. Services contain business rules. Repositories handle database access through Spring Data JPA. DTOs define API input and output. Common errors are handled centrally."

20. How To Explain Security

"The API uses JWT authentication. Login returns a signed token. The JWT filter validates the token on protected requests and sets the authenticated user in Spring Security. Admin-only endpoints use method-level authorization with @PreAuthorize."

21. How To Explain Business Logic

"The most important rules are in the service layer: tickets can only move forward, DONE tickets cannot be edited, tickets with unresolved blockers cannot be completed, deleted tickets and projects are hidden, and mentions are recalculated whenever a comment changes."

22. How To Explain Automation

"Two automated behaviors exist. Auto-assignment chooses the least-loaded developer when a ticket is created without an assignee. Auto-escalation is scheduled and promotes overdue ticket priorities without changing their status."

23. What To Improve In Production

This is a useful interview answer because it shows that you understand the difference between a homework implementation and production hardening.

- Add Flyway or Liquibase migrations instead of relying on Hibernate ddl-auto=update.
- Add a project membership model if the product needs users explicitly linked to projects.
- Add OpenAPI/Swagger documentation.
- Add more controller-level integration tests.
- Consider refresh tokens and stronger token lifecycle management.
- Disable verbose SQL logging in production.

24. One-Sentence Summary To Memorize

"IssueFlow is a secured Spring Boot ticket-management backend with JWT auth, project and ticket workflows, comments, mentions, dependencies, attachments, CSV import/export, soft delete, audit logs, auto-assignment, auto-escalation, and focused tests around the key business rules."

25. Suggested Demo Flow

- Create an admin user with POST /users.
- Login with POST /auth/login and copy the JWT token.
- Call GET /auth/me with Authorization: Bearer token.
- Create a developer user.
- Create a project using ownerId.
- Create a ticket without assigneeId and show auto-assignment.
- Call GET /projects/{id}/workload and show workload sorting.
- Add a comment with @username and show mentionedUsers.
- Call GET /users/{id}/mentions.
- Call GET /audit-logs to show history.